

Functional Programming

CSCI 1101

2024-05-02

Functions as Data

Treating functions as data lets us manipulate them just like any other data.

This enables a style of programming where functions take center stage, known as **functional programming**.

Central to functional programming are functions that act on other functions, **higher-order functions**.

Last time we met some higher-order functions that take functions as arguments.

Today we will meet some that return functions as results.

Function That Return Functions

Here is a function that returns another function:

```
def add(m) :  
    ''' sig: (int) --> fun (int -> int) '''  
    def add_m(n) :  
        ''' sig: (int) --> int '''  
        return m + n  
    return add_m
```

To add two numbers, we give the first number to the add function and the second number to the function that we get as a result of calling the add function:

```
> add(3)(4)  
7
```

Returning Function Literals

Often it is easier to use **function literals** instead of **inner functions** to define the functions that we want to return:

```
def add(m) :  
    ''' sig: (int) --> fun (int -> int) '''  
    return lambda n : m + n
```

This function does exactly the same thing as the last one.

We could even write the whole thing as a function literal:

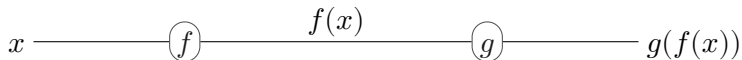
```
''' fun (int -> fun (int -> int)) '''  
add = lambda m : lambda n : m + n
```

Function Composition

If we have two functions where the *result type* of the first is the same as the *argument type* of the second, then we can combine them into a new function by **function composition**:

```
def compose(f , g) :  
    ''' sig: (fun (a -> b) , fun (b -> c)) --> fun (a -> c) '''  
    return lambda x : g(f(x))
```

The *composite function* applies the first function to the argument and the second function to the result:



Composing Functions

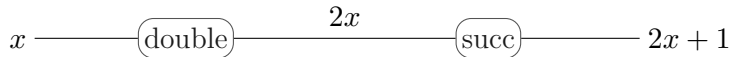
Let's give ourselves some composable functions to work with:

```
double = lambda x : 2 * x
```

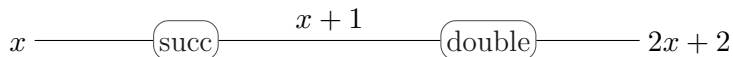
```
succ = lambda x : x + 1
```

Function composition lets us define new functions without using `def` or `lambda`:

```
two_x_plus_one = compose(double , succ)
```



```
two_x_plus_two = compose(succ , double)
```



Composition order matters!

The Identity Function

The **identity function** just gives back its argument as its result:

```
def identity(x) :  
    ''' sig: (a) --> a '''  
    return x
```



This may seem pretty boring, but it's useful when we need to provide a function, but don't want to change anything.

```
two_x = compose(double , identity)
```

Iterating Functions

Using `compose`, `identity` and recursion, we can write a higher-order function that composes a given function with itself a given number of times:

```
def iterate(n , f) :  
    ''' sig: (int , fun (a -> a)) --> fun (a -> a) '''  
    if n > 0 :  
        return compose(f , iterate(n - 1 , f))  
    else :  
        return identity
```

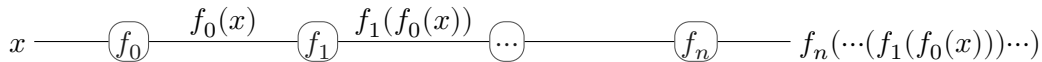


```
eight_x = iterate(3 , double)  
one_x = iterate(0 , double)
```


Composing Lists of Functions

Similarly, we can compose together a list of functions, so long as the result type of each is the same as the argument type of the next:

```
def compose_all(fs) :  
    if fs != [] :  
        return compose(fs[0] , compose_all(fs[1: ]))  
    else :  
        return identity
```



We can think of this as specifying a data processing **pipeline**.

```
four_x_plus_two = compose_all([double, succ, double])  
print_length = compose_all([len, str, print])
```

Partial Application of Functions

We can transform a function that expects two arguments:

```
''' fun (int , int -> int) '''  
mult = lambda x , y : x * y
```

into a function that wants the first argument and returns a function that wants the next argument, by calling it inside of two lambdas:

```
''' fun (int -> fun (int -> int)) '''  
mult_c = lambda x : lambda y : mult(x , y)
```

We can **partially apply** such a function to get another function:

```
> triple = mult_c(3)  # = lambda y : mult(3 , y)
```

And apply *that* function later to get a result:

```
> triple(14)  # = mult(3 , 14)
```

42

Function Currying

Instead of doing this transformation by hand, we can write a higher-order function to do it for *any* two-argument function. This is called **function currying**:

```
def curry(f) :  
    ''' sig: (fun (a , b -> c)) --> fun (a -> fun (b -> c)) '''  
    return lambda x : lambda y : f(x , y)
```

It transforms any function that wants two arguments at once into a function that wants only the first argument and returns a function that wants the second argument.

In **functional programming languages** functions are curried by default.

Function Specialization

By partially applying curried functions we can *specialize* them to a particular task:

```
> triple = curry(mult)(3)
```

```
> # : fun (int -> int)
```

```
> triple(14)
```

```
42
```

```
> double_all = curry(map)(double)
```

```
> # : fun (list int -> list int)
```

```
> list(double_all(range(6)))
```

```
[0, 2, 4, 6, 8, 10]
```

```
> just_evens = curry(filter)(lambda n : n % 2 == 0)
```

```
> # : fun (list int -> list int)
```

```
> list(just_evens(range(6)))
```

```
[0, 2, 4]
```

Activity: Functional Programming

Use higher-order functions to write a function that returns the length of the longest string in a nonempty list of strings.

Your function should not contain the keyword `def`, nor any loops nor recursion.

It should just be an expression:

```
# sig: fun (list str -> int)
longest_string = # your expression goes here
```

such that, for example:

```
> longest_string(str.split("the quick brown fox"))
5
```

Program Functionally

Functions are the heart of programming.

By learning how to get them to do what you want
you can become a more effective programmer.

To Do

Finish *Project 4: Bioinformatics* on CodeRunner in by 2024-05-08.